Panaversity Assignment Report: AI-Assisted App Development

1.

Project Title

Interactive CT Dose Index (CTDIvol) & Dose Length Product (DLP) Estimator

2.

App/Game Idea & Purpose of the Project

- **The Idea:** An interactive desktop and web application designed for medical imaging environments to quickly compute radiation metrics from computed tomography (CT) scans.
- **The Purpose:** In busy clinical settings, manually looking up tissue conversion coefficients (k-factors) and multiplying dose indexes leaves room for calculation errors. This tool automates the math to provide an immediate calculation of patient exposure to help students and techs ensure safety guidelines (ALARA - As Low As Reasonably Achievable) are maintained.

3.

Which AI Tools Were Used

- Gemini was utilized for UI structuring, cross-platform code translation, and syntax troubleshooting.

4.

Initial Prompt Used

"Act as an expert frontend web developer and Medical Physics instructor. Help me create a simple, single-file web application (HTML, CSS, and JavaScript or python) for my university assignment. The app is a 'CT Dose Index (CTDI) & Dose Length Product (DLP) Estimator' to help medical physics students and radiology techs estimate patient radiation risk."

5.

Prompt Iterations Used During Development

- **Iteration 1 (Language Swap):** *"I am a medical physics student and want this tool to showcase backend script handling. Can we convert this entire structure into a standalone Python file instead of HTML?"*
- **Iteration 2 (Layout Tuning):** *"The UI fields look a bit messy. Can we use a grid format inside a custom Tkinter Frame to line up the input text labels cleanly right next to the data boxes?"*

6.

What Worked Well

- Implementing standard ICRP (International Commission on Radiological Protection) k-factors as a strict Python dictionary made mapping the drop-down menu choices seamless.
- Using built-in GUI libraries (`tkinter`) allowed the application to execute as a lightweight desktop tool without forcing the user to install any external modules.

7.

What Did Not Work / Bugs & Challenges Faced

- **The Hidden Extension Trap:** When saving the script via plain text editors, the operating system appended a hidden `.txt` string to the file (saving it as `main.py.txt`). This prevented the script from compiling or running in Python.
- **String Input Errors:** During testing, typing a non-numeric character or accidental space inside the text fields threw a fatal `ValueError` crash inside the program.

8.

How the Issues Were Solved

- **Extension Fix:** Configured the window viewer settings to expose file name extensions, enabling a direct manual rename to strip away the trailing `.txt` syntax.
- **Crash Prevention:** Wrapped the entry query captures inside a rigorous Python `try/except` block. If invalid characters are detected, it catches the `ValueError` and cleanly surfaces a helpful alert pop-up box (`messagebox.showerror`) instead of breaking the app.

9.

Features Added or Improved

- Automated numerical formatting to clamp long outputs to exactly two and three decimal spaces for clinical clarity (e.g., matching standard mSv radiation report formats).
- Added a clear legal and clinical disclaimer at the bottom specifying the tool is a learning prototype.

10.

Final Result of the Project

- A fully operational, responsive, and secure desktop application package layout that safely processes scanning inputs, identifies target anatomical data, and outputs exact tracking metrics instantly.

11.

Does the Project Solve a Real-World Problem?

- **Yes.** It directly impacts healthcare quality assurance and radiation optimization tracking. By providing an instant conversion from machine outputs (CTDI_{vol}) to biological risk impact variables (mSv), it saves operational time and mitigates human math errors in diagnostic imaging departments.